

Übung 4

Hinweis: Verwenden Sie für die folgende Aufgabe keine Lambda-Ausdrücke, Methodenreferenzen oder Streams! Wir haben Sie noch nicht behandelt.

a) Bank

Die Klassen Konto, Girokonto, Sparbuch usw. kennen Sie ja schon. Sie sollen nun eine Klasse `bankprojekt.verwaltung.Bank` schreiben, die diese Konten (und später vielleicht weitere Kontoarten) verwaltet.

Sie soll über folgende öffentliche Methoden/Konstruktoren verfügen:

- `public Bank(long bankleitzahl)`
erstellt eine Bank mit der angegebenen Bankleitzahl
- `public long getBankleitzahl()`
liefert die Bankleitzahl zurück
- `public long girokontoErstellen(Kunde inhaber)`
erstellt ein Girokonto für den angegebenen Kunden. Dabei soll die Methode eine beliebige neue, noch nicht vergebene Kontonummer erzeugen, das neue Girokonto mit dieser Nummer in der Kontenliste speichern und die vergebene Kontonummer zurückgeben.
 - Eine neue, noch nie vergebene Kontonummer ist eine wichtige Aufgabe der Bank. Haben Sie eine Idee, wie Sie das hinkriegen.
- `public long sparbuchErstellen(Kunde inhaber)`
genauso mit Sparbuch
- `public String getAlleKonten()`
liefert eine Auflistung von Kontonummer und Kontostand zu jedem Konto in jeweils einer Zeile zurück
- `public Set<Long> getAlleKontonummern()`
liefert eine Menge aller gültigen Kontonummern in der Bank
- `public SortedSet<Kunde> getAlleKunden()`
liefert eine Menge aller Kunden, die bei dieser Bank mindestens ein Konto haben. Die Kunden sollen dabei absteigend nach ihrem Geburtstag sortiert sein.
 - Tipp: Eine `TreeSet` ist eine `SortedSet`. Schauen Sie sich die Konstruktoren an.
- `public boolean kontoLoeschen(long nummer)`
löscht das Konto mit der angegebenen `nummer` und gibt `true` zurück, wenn tatsächlich ein Konto gelöscht wurde, bzw. `false`, wenn die Kontonummer gar nicht existierte.
- `public Geldbetrag getKontostand(long nummer)`
liefert den Kontostand des Kontos mit der angegebenen `nummer` zurück.
- `public boolean geldAbheben(long von, Geldbetrag betrag)`
hebt den Betrag vom Konto mit der Nummer `von` ab und gibt zurück, ob die Abhebung geklappt hat.
- `public void geldEinzahlen(long auf, Geldbetrag betrag)`
zahlt den angegebenen Betrag auf das Konto mit der Nummer `auf` ein
- `public boolean geldUeberweisen(long vonKontonr, long nachKontonr, Geldbetrag betrag, String verwendungszweck)`
überweist (das ist etwas anderes als Geld abheben und wieder einzahlen...) den ge-

nannten Betrag vom überweisungsfähigen Konto mit der Nummer `vonKontonr` zum überweisungsfähigen Konto mit der Nummer `nachKontonr` und gibt zurück, ob die Überweisung geklappt hat (nur bankinterne Überweisungen!) Überlegen Sie gut, was dabei alles schief gehen kann! (Vorsicht: Geld darf nicht einfach „verschwinden“.)

- `public Map<Kunde, Geldebtrag> getGesamtkontostaende()`
liefert eine Map zurück, in der für jeden Kunden der Bank die Summe der Kontostände aller seiner Konten angegeben ist.
- `public int kontenEinesKundenLoeschen(Kunde inhaber)`
löscht alle Konten, die `inhaber` gehören, und gibt die Anzahl der gelöschten Konten zurück.

Wenn Sie möchten, können Sie die Signaturen der Methoden auch etwas abändern, um auf die unterschiedlichen Fehlersituationen hinzuweisen, z.B. könnte eine `KontonummerNichtVorhandenException` (oder so) geworfen werden.

Ein formaler Test ist (in dieser Übung noch) nicht notwendig. Zum Ausprobieren steht aber die Klasse `Bankspielereien` zur Verfügung, in der Sie Ihre Methoden aufrufen können.

Packen Sie mit dem Maven-assembly-Plugin alle **Quelltextdateien** und die `pom.xml` in ein jar-Archiv und laden Sie dieses Archiv in Moodle hoch. Bitte geben Sie die Übungsaufgaben ab jetzt immer in dieser Form ab.